

Big Data and Automated Content Analysis (6 ECTS)

Cursusdossier

dr. Anne Kroon
Graduate School of Communication
University of Amsterdam
a.c.kroon@uva.nl

Academic Year 2026/27
Semester 1, block 2

Contents

1	Short description of the course	2
2	Exit qualifications	4
3	Testable objectives	6
4	Planning of testing and teaching	8
5	Literature	10
6	Specific course timetable	12
7	Testing	19
8	Lecturers' team, including division of responsibilities	22
9	Calculation of students' study load (in hours)	23
10	Calculation of lecturers' teaching load (in hours)	24
11	History of the course	25
A	Final Individual Project	27

Chapter 1

Short description of the course

“Big data” refers to data that are more voluminous, but often also more varied, unstructured, and dynamic than data traditionally used in communication research. In Communication Science and the Social Sciences more broadly, this includes research using internet-based data sources such as social-media content, digital news archives, platform data, and public comments. This area of research is commonly described as *Computational Social Science* (Lazer et al., 2009) or, when focused on communication processes and content, *Computational Communication Science* (Shah, Cappella, & Neuman, 2015).

This course introduces the concepts, opportunities, and challenges of computational research with digital communication data. Computational approaches can make it possible to analyse large or complex collections of texts and other digital traces; they complement rather than replace established communication-science methods such as theory-driven research design, manual content analysis, and statistical inference. Throughout the course, we emphasise that automated outputs require substantive interpretation and empirical validation.

Participants learn practical strategies for acquiring, organising, preprocessing, analysing, and documenting digital data in communication contexts. The course focuses on (a) data acquisition from files and APIs, data wrangling, exploratory analysis, and reproducible workflows; and (b) automated content analysis, including text preprocessing, dictionary and Bag-of-Words approaches, unsupervised text analysis, embeddings, topic modelling, and supervised machine-learning classification. Students learn to inspect and evaluate automated outputs, compare them with appropriate benchmarks, and reflect on validity, ethics, reproducibility, and the responsible use of AI tools.

The course uses Python as its main programming environment. Students

are expected to be willing to learn to write, read, adapt, and explain Python code. Previous programming experience is helpful but not required. Students without prior experience are encouraged to complete one of the many free online introductions to Python before the course begins.

Chapter 2

Exit qualifications

The course contributes to the following three exit qualifications of the Research Master's programme in Communication Science:

Expertise in empirical research

3. Knowledge and Understanding: Have in-depth knowledge and a thorough understanding of advanced research designs and methods.
4. Skills and abilities: Are able, independently and on their own, to set up, conduct, report, and interpret advanced academic research.

Academic abilities and attitudes

6. Academic attitudes: Accept that scientific knowledge is always 'work in progress' and that arguments must be considered and conclusions drawn on the basis of empirical results and valid criticism.

The course contributes to these exit qualifications at the level of *fundamental* techniques of Big Data and Automated Content Analysis. In particular, students learn to examine their own assumptions critically, justify methodological choices, acknowledge limitations and uncertainties, submit their work to critical discussion, and respond constructively to valid criticism. These academic attitudes are assessed summatively through the individual project pitch and the final individual project.

The exit qualifications are elaborated in the following specifications:

3. Knowledge and Understanding: Have in-depth knowledge and a thorough understanding of advanced research designs and methods.
 - 3.1. Have in-depth knowledge and a thorough understanding of advanced research designs and methods, including their value and limitations.

- 3.2. Have in-depth knowledge and a thorough understanding of advanced techniques for data analysis.
- 4. Skills and abilities: Are able, independently and on their own, to set up, conduct, report, and interpret advanced academic research.
 - 4.1. Are able to formulate research questions and hypotheses for advanced empirical studies.
 - 4.2. Are able to develop a research plan, choose appropriate and suitable research designs and methods for advanced empirical studies, and justify the underlying choices.
 - 4.3. Are able to assess the validity and reliability of advanced empirical research, and to judge the scientific and professional value of findings from advanced empirical research.
 - 4.4. Are able to apply advanced empirical research methods.
- 5. Academic attitudes.
 - 6.1. Regularly assess their own assumptions, strengths, and weaknesses critically.
 - 6.2. Accept that scientific knowledge is always ‘work in progress’ and that something regarded as ‘true’ may be proven to be false, and vice versa.
 - 6.3. Are keen to acquire new knowledge, skills, and abilities.
 - 6.4. Are willing to share and discuss arguments, results, and conclusions, including submitting one’s own work to peer review.
 - 6.5. Are convinced that academic debates should not be conducted on the basis of rhetorical qualities but that arguments must be considered and conclusions drawn on the basis of empirical results and valid criticism.

Chapter 3

Testable objectives

3. Knowledge and Understanding: Have in-depth knowledge and a thorough understanding of advanced research designs and methods.

3.1. Have in-depth knowledge and a thorough understanding of advanced research designs and methods, including their value and limitations.

3.2. Have in-depth knowledge and a thorough understanding of advanced techniques for data analysis.

A Students can explain *fundamental research designs* and *commonly employed methods* in existing research articles on Big Data and automated content analysis.

B Students can on their own and in own words critically discuss the pros and cons of *fundamental* research designs and methods employed in existing research articles on Big Data and automated content analysis; they can, based on this, give a critical evaluation of the methods and, where relevant, give advice to improve the study in question.

C Students are able to identify some *basic* techniques from the field of computer science and computer linguistics that are applicable to research in communication science; they can explain the principle of some traditional approaches to text analysis, namely simple rule-based techniques and basic methods of unsupervised and supervised machine learning.

4. Skills and abilities: Are able, independently and on their own, to set up, conduct, report and interpret advanced academic research.

4.1 Are able to formulate research questions and hypotheses for advanced empirical studies.

4.2 Are able to develop a research plan, choose appropriate and suitable research designs and methods for advanced empirical studies, and justify the underlying choices.

4.3 Are able to assess the validity and reliability of advanced empirical research, and to judge the scientific and professional value of findings from advanced empirical research.

4.4 Are able to apply advanced empirical research methods.

- D Students can on their own formulate a research question and hypotheses for own empirical research in the domain of Big Data.
 - E Students can on their own chose, execute and report on *fundamental* research methods in the domain of Big Data and automatic content analysis.
 - F Students know how to collect data with APIs or read in existing data files; they know how to analyze these data with fundamental automated techniques and to this end, they have basic knowledge of the programming language Python and know how to use fundamental Python-modules for communication science research.
6. Academic attitudes
- 6.1 Regularly assess their own assumptions, strengths and weaknesses critically.
 - 6.2 Accept that scientific knowledge is always 'work in progress' and that something regarded as 'true' may be proven to be false, and vice-versa.
 - 6.3 Are keen to acquire new knowledge, skills and abilities.
 - 6.4 Are willing to share and discuss arguments, results and conclusions, including submitting one's own work to peer review.
 - 6.5 Are convinced that academic debates should not be conducted on the basis of rhetorical qualities but that arguments must be considered and conclusions drawn on the basis of empirical results and valid criticism.
- G Students can critically discuss strong and weak points of their own research using fundamental techniques from the field of Big Data and Automated Content Analysis, and suggest improvements.
 - H Students can critically reflect on the assumptions, strengths, limitations, and potential improvements of their own planned research using fundamental techniques from the field of Big Data and Automated Content Analysis. They can defend their methodological choices using empirical and methodological arguments and respond constructively to valid criticism.

Chapter 4

Planning of testing and teaching

The course consists of seven teaching weeks with two meetings per week, followed by an assessment period. Generally, the first weekly meeting combines short lectures on key concepts and methods with seminar-style discussions of theoretical considerations, methodological choices, and examples from the literature. Exceptions are the individual in-class coding exercise on Monday 16 November and the individual project pitches on Monday 7 December. The second weekly meeting generally consists of a practical lab session in which students apply the techniques they have learned to course materials or their own project data. Exceptions are the lecture on textual data on Thursday 19 November and individual consultation on Thursday 10 December. The lab on Thursday 12 November also includes a short lecture introducing text as data.

During lab sessions, collaboration and mutual support are strongly encouraged. Students are encouraged to discuss programming concepts, compare approaches, ask questions, and help one another work through exercises. This collaboration supports learning but does not permit students to share or jointly produce code, analyses, or written material for individual assessments.

The course contains three individual summative assessments:

- an in-class coding exercise in Week 4 (25%);
- an individual project pitch in Week 7 (10%);
- a final individual project (65%).

The Week 4 coding exercise assesses students' ability to independently read, adapt, test, debug, and explain Python code. The Week 7 project

pitch assesses the feasibility and methodological justification of the proposed research plan. It also assesses students' ability to identify and explain a relevant limitation, risk, or methodological weakness in their proposed study and to propose a feasible improvement. This provides individual summative evidence for ILO H. The pitch further enables the lecturer to assess students' understanding of their proposed data, methods, and analytical choices verbally. During the Monday pitch session, students are encouraged to provide constructive feedback to peers. This peer-feedback activity is formative and is not graded. The Thursday meeting is reserved for individual consultation.

Generative-AI safeguards

The in-class coding exercise is completed individually without the use of generative-AI tools, subject to the applicable examination arrangements. In other course work, generative AI may be used only to clarify programming concepts or debug small, de-identified code snippets or toy examples. Students may not upload course datasets, substantial project data, full analysis notebooks, or sensitive or personally identifiable data to external AI services. They may not use generative AI as a black-box analysis engine for graded work.

Students remain responsible for the correctness, efficiency, reproducibility, and documentation of all submitted code. Non-trivial generative-AI contributions to code must be disclosed in the relevant notebook or script.

Chapter 5

Literature

The following schedule gives an overview of the topics covered each week, the obligatory literature that has to be studied each week, and other tasks the students have to complete in preparation of the class. In particular, the schedule shows which chapter of van Atteveldt, Trilling, and Arcila Calderón (2022) will be dealt with. Note that some basic chapters that explain how to install the software we are going to use have to be read before the course starts.

Next to the obligatory literature, the following books provide the interested student with more and deeper information. They are intended for the advanced reader and might be useful for final individual projects, but are by no means required literature. Bear in mind, though, that you may encounter slightly outdated examples (e.g., Python 2, now-defunct APIs etc.).

- McKinney, 2012: A lot of examples for data analysis in Python. A HTML-based version of the book can be viewed for free at <https://wesmckinney.com/book/>.
- VanderPlas, 2016: A more recent book on numpy, pandas, scikit-learn and more. It can also be read online for free on <https://jakevdp.github.io/PythonDataScienceHandbook/>. The contents are available as Jupyter Notebooks as well, see <https://github.com/jakevdp/PythonDataScienceHandbook>.
- The pandas cookbook by Julia Evans, a collection of notebooks on github: <https://github.com/jvns/pandas-cookbook>.
- Hovy, 2020: A thin book on bottom-up text analysis in Python with both a bit more math background and ready-to-use Python code implementations.

- Salganik, 2017: Not a book on Python, but on research methods in the digital age. Very readable, and a lots of inspiration and background about techniques covered in our course.

Chapter 6

Specific course timetable

Before the course starts: Prepare your computer.

✓ **CHAPTER 1: INTRODUCTION**

Make sure that you have a working Python environment installed on your computer before the course begins. Detailed installation instructions will be provided in advance. Students are responsible for bringing a laptop with a functioning Python environment to every meeting.

Week 1: Programming for Computational (Communication|Social) Scientists

Monday, 26–10. Lecture, 15:00–17:00

✓ **KITCHIN (2014)**

✓ **HILBERT ET AL. (2019)**

We discuss what Big Data and Computational (Social|Communication) Science are. We consider their opportunities and challenges, including implications for social-scientific research. We also introduce the tools used in Computational Social Science, particularly Python.

Additionally, the journal *Communication Methods and Measures* published a special issue (volume 12, issue 2–3) on Computational Communication Science. You are invited to read at least the editorial (van Atteveldt & Peng, 2018); other articles can be read later in the course.

Towards the end of the lecture, we will make first contact with writing and running Python code. We will also discuss expectations concerning

reproducibility, responsible use of external code and AI tools, and active participation in the course.

Thursday, 29–10. Lab session, 15:00–17:00

- ✓ CHAPTER 3: PROGRAMMING CONCEPTS FOR DATA ANALYSIS
- ✓ CHAPTER 4: HOW TO WRITE CODE

You will receive a gentle introduction to computer programming. During the lab, you are encouraged to follow examples on your own laptop. We will take the first steps in Python and complete exercises on variables, data types, control flow, functions, and debugging.

Week 2: From files and APIs to lists, dictionaries, or dataframes

We discuss file formats such as `csv` and `json`; encodings; reading files into basic Python structures such as dictionaries and lists; reading data into dataframes; and retrieving data from local files, packages, and APIs. We also consider data provenance and the practical limits of data access.

Monday, 02–11. Lecture, 15:00–17:00

- ✓ CHAPTER 5: FROM FILE TO DATAFRAME AND BACK
- ✓ FREELON (2018-10)

This lecture provides a conceptual overview of different file formats and data sources, alongside practical guidance on handling data in basic Python and pandas. We introduce APIs as a means of data collection and discuss why documentation, metadata, and collection decisions matter for later analysis.

Thursday, 05–11. Lab session, 15:00–17:00

We will practise using the data structures introduced in Week 1 and work with different file formats. Exercises include inspecting data, handling encodings, reading nested JSON structures, and extracting information from a provided API response.

Week 3: Data wrangling and exploratory data analysis

You do not need Python to conduct conventional statistics. However, using a single environment for data acquisition, cleaning, analysis, and visualisation makes a workflow more transparent and reproducible. This week, we discuss how to organise data, including long and wide formats, and how to conduct exploratory analyses and create simple visualisations in Python.

Monday, 09–11. Lecture, 15:00–17:00

- ✓ CHAPTER 6: DATA WRANGLING
- ✓ CHAPTER 7.1: SIMPLE EXPLORATORY DATA ANALYSIS
- ✓ CHAPTER 7.2: VISUALISING DATA

We will learn how to prepare data for analysis and develop a first understanding of a dataset through exploratory analysis and visualisation. We cover data wrangling with pandas, including filtering, aggregating, reshaping between long and wide formats, joining datasets, and handling missing values.

Thursday, 12–11. Lab session and short lecture, 15:00–17:00

We will apply the techniques from the previous lectures to multiple datasets. The session also prepares students for the individual in-class coding exercise in Week 4. We conclude with a short introduction to text as data, including tokenisation and Bag-of-Words representations.

Week 4: Individual in-class coding exercise and introduction to textual data

Monday, 16–11. Individual in-class coding exercise, 15:00–17:00

This individual, on-campus coding exercise covers the foundations from Weeks 1–3. It assesses students' ability to read, adapt, debug, and explain basic Python and data-analysis code and contributes 25% to the final grade.

Within the scheduled session, students have 60 minutes to complete the exercise, followed by a 10-minute submission buffer for saving, renaming, and uploading the notebook. The exercise is open book: students may consult lecture slides, readings, course notebooks, personal notes, and Python/pandas documentation saved on their computer or printed before the exercise. Internet access is permitted only to download the assignment and upload the completed notebook. Browsing websites, searching online, downloading other materials during the exercise, using generative-AI tools, and communicating with or receiving help from others are not permitted.

Thursday, 19–11. Lecture: Processing textual data, 15:00–17:00

- ✓ CHAPTER 9: PROCESSING TEXT
- ✓ CHAPTER 10: TEXT AS DATA
- ✓ CHAPTER 11, SECTIONS 11.1–11.3: AUTOMATIC ANALYSIS OF TEXT

Additional recommended background reading on stopwords: Nothman, Qin, and Yurchak (2018).

This session introduces text processing as a measurement problem. We explore tokenisation, lemmatisation, stopword removal, n-grams, word counts, word co-occurrences, and regular expressions. We implement selected techniques in Python and discuss how preprocessing choices affect the validity of automated content analysis.

Week 5: Unsupervised approaches to text analysis

This week, we transition from transparent frequency-based text representations to unsupervised machine-learning approaches. We discuss how traditional topic models relate to embedding-based methods and how these approaches can support exploratory text analysis.

Monday, 23–11. Lecture: From topic models to embeddings, 15:00–17:00

- ✓ CHAPTER 11.5: UNSUPERVISED TEXT ANALYSIS — TOPIC MODELLING AND BEYOND
- ✓ KROON, WELBERS, TRILLING, AND VAN ATTEVELDT (2024)

✓ MAIER ET AL. (2018)

We examine the logic of unsupervised text modelling and contrast traditional approaches such as LDA with modern embedding-based approaches. We introduce BERTopic as an example of a workflow that uses embeddings, dimensionality reduction, clustering, and topic descriptions.

We emphasise that model outputs are candidate patterns rather than established findings: they require substantive interpretation, inspection of representative documents, and evaluation of their robustness and limitations.

Thursday, 26–11. Lab session, 15:00–17:00

We apply an unsupervised text-analysis workflow to a textual dataset. We compare output from simpler approaches with embedding-based clustering and discuss interpretation, outliers, parameter choices, reproducibility, and limits of automated topic descriptions.

Week 6: Supervised approaches to text analysis

This week introduces the logic of supervised machine learning for automated content analysis. We focus on classification tasks, evaluation, and the difference between a technically functioning model and a valid measurement of a communication-science construct.

Monday, 30–11. Lecture, 15:00–17:00

✓ CHAPTER 8: STATISTICAL MODELING AND SUPERVISED MACHINE LEARNING

✗ STUDENTS MAY SKIP SECTION 8.4: DEEP LEARNING

✓ BOUMANS AND TRILLING (2016)

We discuss supervised machine learning, labelled data, training and test data, classification, and performance evaluation. We consider accuracy, precision, recall, F1, and the importance of inspecting model errors. We also discuss how dictionary methods, conventional classifiers, embedding-based approaches, and LLM-assisted classification can be compared and validated.

LLM-assisted classification is discussed conceptually; the lab uses scikit-learn. This discussion does not authorise the use of generative AI as an

analysis engine for graded work. Any practical use of LLM-assisted classification requires explicit authorisation from the lecturer and must comply with the course rules on data sharing and disclosure.

Thursday, 03–12. Lab session, 15:00–17:00

We work with scikit-learn to construct and evaluate a basic text classifier. Students will compare automated outputs with manually coded examples, inspect errors, and discuss what evidence would be needed to support a substantive interpretation.

Week 7: Project pitches, peer feedback, and consultation

Monday, 07–12. Individual project pitches and peer feedback, 15:00–17:00

Each student presents a short pitch for the final project. The pitch should address:

- the research question and substantive motivation;
- the planned dataset;
- the intended analytical strategy;
- preliminary data inspection or preliminary output;
- one anticipated issue concerning validity, ethics, reproducibility, or AI use, together with a feasible response or improvement.

The individual pitch contributes 10% to the final grade. After presenting, each student answers critical questions about the proposed data, methods, assumptions, and limitations. Students are encouraged to discuss project plans and offer constructive feedback to peers. This discussion supports learning and is not separately graded.

Thursday, 10–12. Individual consultation, 15:00–17:00

Students can consult individually with the lecturer about methodological, practical, ethical, or reproducibility questions concerning their final project.

Week 8: Assessment week

Final individual project

Deadline for handing in: Wednesday, 16–12, 23:59.

Chapter 7

Testing

An overview of the testing is given in the Test Matrix, displayed in Table 7.1.

Grading

The final grade for this course is composed of an individual in-class coding exercise (25%), an individual project pitch (10%), and a final individual project (65%).

In-class coding exercise (25%)

The Week 4 in-class coding exercise assesses students' individual ability to work with Python-based data-analysis workflows and their understanding of the fundamental concepts introduced during the first part of the course. Students are asked to read, adapt, test, debug, and/or explain code using fundamental techniques taught in the course. The exercise also includes short questions about relevant course literature. The exercise is completed individually and without the use of generative-AI tools.

Grading criteria are communicated to students together with the assignment. In general, the assessment considers:

- students' understanding of Python basics and fundamental course concepts;
- their ability to work independently with basic data-analysis code;
- the correctness and clarity of their code, explanations, and interpretations; and

Table 7.1: Test Matrix

	In-class coding exercise	Individual project pitch	Final individual project
	<i>25% of final grade</i>	<i>10% of final grade</i>	<i>65% of final grade</i>
A. Students can explain fundamental research designs and commonly employed methods in existing research articles on Big Data and automated content analysis.	X		X
B. Students can critically discuss the pros and cons of fundamental research designs and methods employed in existing research articles on Big Data and automated content analysis, evaluate these methods, and, where relevant, advise on improvements.		X	X
C. Students can identify basic techniques from computer science and computational linguistics that are applicable to communication-science research and explain the principles of rule-based techniques and basic unsupervised and supervised machine-learning methods.			X
D. Students can independently formulate a research question and hypotheses for empirical research in the domain of Big Data.		X	X
E. Students can independently choose, execute, and report on fundamental research methods in the domain of Big Data and automated content analysis.	X	X	X
F. Students can collect data with APIs or read existing data files, analyse these data with fundamental automated techniques, and use fundamental Python modules for communication-science research.	X		X
G. Students can critically discuss strong and weak points of their own research using fundamental techniques from the field of Big Data and Automated Content Analysis, and suggest improvements.		X	X
H. Students can critically reflect on the assumptions, strengths, limitations, and potential improvements of their own planned research using fundamental techniques from the field of Big Data and Automated Content Analysis. They can defend their methodological choices using empirical and methodological arguments and respond constructively to valid criticism.		X	

- the quality of their responses to questions related to relevant course literature and methods.

Individual project pitch (10%)

In Week 7, students give an individual pitch of their proposed final project. The pitch assesses the feasibility and methodological justification of the proposed research plan. It also assesses students' ability to justify methodological choices, acknowledge a relevant limitation or risk in the planned research, and respond constructively to critical questions.

Grading criteria are communicated to students together with the assignment. In general, the assessment considers:

- the clarity and feasibility of the proposed research project;
- the justification of the proposed data, research design, and analytical strategy; and
- the student’s critical reflection on limitations or risks and ability to answer questions about the proposed work.

Final individual project (65%)

The final individual project is an individual computational communication-science research project. It accounts for 65% of the final grade and assesses students’ ability to formulate and answer a research question using fundamental techniques taught in the course.

Detailed requirements concerning the report, project files, critical reflection, submission, attribution, and permitted generative-AI use are provided in Appendix A.

Grading and second attempt

Students must receive a passing grade (5.5 or higher) for each of the three individual assessment components: the in-class coding exercise, the individual project pitch, and the final individual project. If the grade for one component is lower than 5.5, the student may complete one reassessment opportunity for that component, in the form and by the deadline communicated by the lecturer.

A reassessment of the in-class coding exercise is completed individually in a supervised setting and without generative-AI tools. A reassessment of the project pitch takes the form of an individual pitch or oral assessment. An improved version of the final individual project may not be graded higher than 6.0. If a reassessment is graded lower than 5.5, the course cannot be completed.

Chapter 8

Lecturers' team, including division of responsibilities

The course is taught and coordinated by Dr. Anne Kroon. She is responsible for all aspects of the course, including teaching, coordination, and the grading of assessments.

Chapter 9

Calculation of students' study load (in hours)

- Elective total: 6 ECTS = 168 hours.
- Reading and preparation:
 - Selected textbook chapters and articles: approximately 280 pages.
At six pages per hour, this amounts to 46.6 hours.
 - Reading and completing tutorials to acquire practical skills: 40 hours.
 - Reading and preparation total: 86.6 hours.
- Presence: 14 meetings of two hours: 28 hours.
- In-class coding exercise, including preparation: 12 hours.
- Individual project pitch, including preparation: 8 hours.
- Final individual project, including data collection, analysis, and write-up: 33.4 hours.

Total: 168 hours.

Chapter 10

Calculation of lecturers' teaching load (in hours)

- Contact hours: 28 hours (14 meetings of two hours).
- Preparation and updating of lectures: 20 hours.
- Preparation and testing of practical exercises: 16 hours.
- Python installation support and other individual assistance outside scheduled meetings: 8 hours.
- Development and grading of the in-class coding exercise: 12 hours.
- Preparation of assessment, feedback, and grading of individual project pitches: 6 hours.
- Grading and feedback on final individual projects: 25 hours.
- Course coordination, including Canvas, communication, planning, and administration: 16 hours.

Total: 131 hours, including course coordination. This allocation assumes reuse and updating of existing teaching materials. Scheduled individual consultation is included in the contact hours.

Chapter 11

History of the course

In response to the feedback by the test review committee in 2019, the following changes were applied:

- Emphasised in the section “Testable Objectives” that—in contrast to the 12 ECTS course—students should be able to demonstrate, reflect, and apply knowledge about *fundamental*, rather than *advanced*, techniques of automated content analysis. Likewise, testing is based on the extent to which students demonstrate knowledge and skills at the level of *fundamental* techniques. This distinguishes the 6 ECTS course from more advanced courses in Big Data and Automated Content Analysis.

In 2022, the course received a thorough update following the publication of the new textbook (van Atteveldt et al., 2022).

In 2023, the course was updated to incorporate recent developments in machine learning. The course continues to focus on fundamental techniques of automated content analysis. In the current course structure, Weeks 5 and 6 include an outlook on state-of-the-art topic-modelling and machine-learning approaches.

In 2024, the course was updated in response to developments in AI-assisted coding. The take-home exam was redesigned to focus more strongly on debugging and explaining existing code, including a short video walk-through. In addition, a final in-class project pitch was introduced to enable students to exchange ideas and provide feedback while there was still time to revise their projects.

In 2025, these changes were maintained and consolidated. The take-home examination continued to require a walkthrough video, and the in-class project presentation was formally integrated into the course structure.

In 2026, the course was revised in response to feedback from the Toetscommissie following its review of the RM Year 2 curriculum. The Toetscommissie

appreciated the variety of lectures, exercises, and lab sessions, the relevance of the skills taught, and the clarity and level of the assignments. It identified two points requiring attention: it was not sufficiently clear how ILO H, concerning academic attitudes, was summatively assessed, or what the earlier designation “precondition” meant; and the course file did not make safeguards concerning generative-AI use and possible “vibe-coding” sufficiently visible. The committee also suggested considering small individual in-class graded coding assignments. In accordance with the new procedure, this feedback and the response to it are recorded explicitly here.

In response, the 2026–2027 edition introduces an individual in-class coding exercise in Week 4, worth 25% of the final grade. The exercise requires students to independently read, adapt, test, debug, and explain Python-based data-analysis workflows. It provides direct evidence of individual programming competence and understanding without reliance on generative-AI tools.

ILO H has been reformulated while remaining closely tied to the programme’s established exit criteria for academic attitudes. It focuses on students’ ability to examine their assumptions critically, defend methodological choices using empirical and methodological arguments, acknowledge limitations and possible improvements, and respond constructively to valid criticism. This outcome is summatively assessed through the individual Week 7 project pitch, worth 10% of the final grade. The pitch rubric requires students to identify and explain a relevant limitation, risk, or methodological weakness in their proposed study, propose a feasible improvement, and answer critical questions. The earlier ungraded “precondition” category has therefore been removed from the test matrix. Peer feedback remains formative rather than being used as indirect evidence for an individual grade.

The course file now includes an explicit policy on generative-AI use. Generative-AI tools are not permitted during the in-class coding exercise. For other course work, students may use such tools only within defined limits; they may not upload course datasets or full analysis notebooks to external services, use AI as a black-box analysis engine, or submit non-trivial AI-assisted code without disclosure. Collaboration and mutual support are strongly encouraged during lab sessions. Peer feedback during the Monday Week 7 pitch session is formative and is not graded; the Thursday meeting is reserved for individual consultation.

Appendix A

Final Individual Project

The final individual project accounts for 65% of the overall course grade. Students design and execute an independent Computational Communication Science project that demonstrates their ability to formulate a research question, collect and analyse data, and apply fundamental techniques taught in the course.

This is an individual assessment. The report and all submitted code must be the student's own work, except where external sources or permitted generative-AI contributions are appropriately disclosed in accordance with the course manual.

Your submission must include:

1. a PDF report; and
2. a `.zip` folder containing all relevant code and output files.

PDF report

The recommended length of the report is five to six pages (including references). It should include the following components.

A. **Introduction** (one to two pages)

- A clearly formulated research question;
- Any subquestions and/or hypotheses;
- References to relevant literature, including course readings where appropriate.

B. **Analytic strategy**

- A transparent overview of the methodological approach;
- A justification for the choice of techniques, referring explicitly to methods discussed during the course.

C. Dataset description

- A carefully collected and relevant dataset of non-trivial size, for example retrieved via APIs or constructed by combining existing datasets;
- An exploratory description of the dataset;
- A brief argument explaining why the dataset is suitable for the research question.

D. Results

- A clear presentation of the analyses and findings needed to answer the research question;
- Appropriate tables, figures, examples, or model-evaluation results;
- A substantive interpretation of the findings that distinguishes empirical results from speculation.

E. Conclusion and critical reflection

- A well-substantiated answer to the research question;
- A critical discussion of the strengths, limitations, and uncertainties of the research design, data, analysis, and conclusions;
- An explanation of the implications of these limitations for the interpretation of the findings, as well as feasible directions for future research.

The conclusion and critical-reflection component contributes to the assessment of students' ability to critically discuss the strengths and weaknesses of their completed research and suggest improvements.

Project files (.zip folder)

Alongside the PDF report, include a compressed folder containing:

- Python scripts or notebooks for data collection, preprocessing, analysis, and visualisation. These files must be well documented, reproducible, and tailored to the student's own project;

- Output files, such as tables, plots, or summary statistics;
- The data needed to reproduce the analysis, where sharing those data is legally, ethically, and contractually permitted;
- A README file describing:
 - how to run the code;
 - what each script or notebook does; and
 - any software or library requirements.

Requirements and guidance

Depending on the chosen topic, students must apply at least one technique taught in the course. Projects should include an exploratory description of the dataset and apply fundamental analytical techniques.

During lab sessions and individual consultations, students may discuss:

- the scope of the project;
- the methodological requirements that the project must meet; and
- how the chosen analytical methods contribute to the final grade.

Important notes

- All external data sources and code, including libraries, templates, and tutorials, must be properly cited.
- Code must be tailored to the student's own project and run without errors.
- Any non-trivial contribution by a generative-AI tool must be disclosed in the relevant notebook or script, as specified in the course manual.
- The rules on generative-AI use in the course manual apply in full. In particular, students may not upload course datasets or full analysis notebooks to external generative-AI services, or use generative AI as a black-box analysis engine for graded work.
- Students are free to select any topic of interest—preferably topics within traditional communication science or the social sciences—provided that it is framed around a research question and based on original data analysis.

Submission instructions

Submit both the PDF report and the `.zip` folder containing code and outputs via Canvas.

Deadline: Submission is open until 16 December, 23:59. Late submissions are handled in accordance with the course manual and applicable examination regulations.

Literature

- Boumans, J. W., & Trilling, D. (2016). Taking stock of the toolkit: An overview of relevant automated content analysis approaches and techniques for digital journalism scholars. *Digital Journalism*, 4(1), 8–23. doi: 10.1080/21670811.2015.1096598
- Freelon, D. (2018-10). Computational research in the post-API age. *Political Communication*, 35(4), 665–668. Retrieved from <https://doi.org/10.1080/10584609.2018.1477506> doi: 10.1080/10584609.2018.1477506
- Hilbert, M., Barnett, G., Blumenstock, J., Contractor, N., Diesner, J., Frey, S., ... Zhu, J. J. H. (2019). Computational Communication Science : A Methodological Catalyzer for a Maturing Discipline. *International Journal of Communication*, 13, 3912–3934.
- Hovy, D. (2020). *Text analysis in Python for social scientists: Discovery and exploration*. Cambridge University Press. doi: 10.1017/9781108873352
- Kitchin, R. (2014). Big Data, new epistemologies and paradigm shifts. *Big Data & Society*, 1(1), 1–12. doi: 10.1177/2053951714528481
- Kroon, A. C., Welbers, K., Trilling, D., & Van Atteveldt, W. (2024, April). Advancing automated content analysis for a new era of media effects research: The key role of transfer learning. *Communication Methods and Measures*, 18(2), 142–162. Retrieved 2024-09-20, from <https://www.tandfonline.com/doi/full/10.1080/19312458.2023.2261372> doi: 10.1080/19312458.2023.2261372
- Lazer, D., Pentland, A., Adamic, L., Aral, S., Barabási, A.-L., Brewer, D., ... van Alstyne, M. (2009). Computational social science. *Science*, 323, 721–723. doi: 10.1126/science.1167742
- Maier, D., Waldherr, A., Miltner, P., Wiedemann, G., Niekler, A., Keinert, A., ... Adam, S. (2018). Applying LDA topic modeling in communication research: Toward a valid and reliable methodology. *Communication Methods and Measures*, 12(2-3), 93–118. doi: 10.1080/19312458.2018.1430754
- McKinney, W. (2012). *Python for data analysis*. O'Reilly.

- Nothman, J., Qin, H., & Yurchak, R. (2018). Stop Word Lists in Free Open-source Software Packages. In *Proceedings of workshop for nlp open source software (nlp-oss)* (pp. 7–12). Association for Computational Linguistics. doi: 10.18653/v1/W18-2502
- Salganik, M. J. (2017). *Bit by bit: Social research in the digital age*. Princeton University Press.
- Shah, D. V., Cappella, J. N., & Neuman, W. R. (2015). Big Data, digital media, and computational social science: Possibilities and perils. *The ANNALS of the American Academy of Political and Social Science*, 659(1), 6–13. doi: 10.1177/0002716215572084
- van Atteveldt, W., Trilling, D., & Arcila Calderón, C. (2022). *Computational analysis of communication: A practical introduction to the analysis of texts, networks, and images with code examples in python and r*. Wiley.
- van Atteveldt, W., & Peng, T. Q. (2018). When Communication Meets Computation: Opportunities, Challenges, and Pitfalls in Computational Communication Science. *Communication Methods and Measures*, 12(2-3), 81–92. doi: 10.1080/19312458.2018.1458084
- VanderPlas, J. (2016). *Python data science handbook: Essential tools for working with data*. O'Reilly.